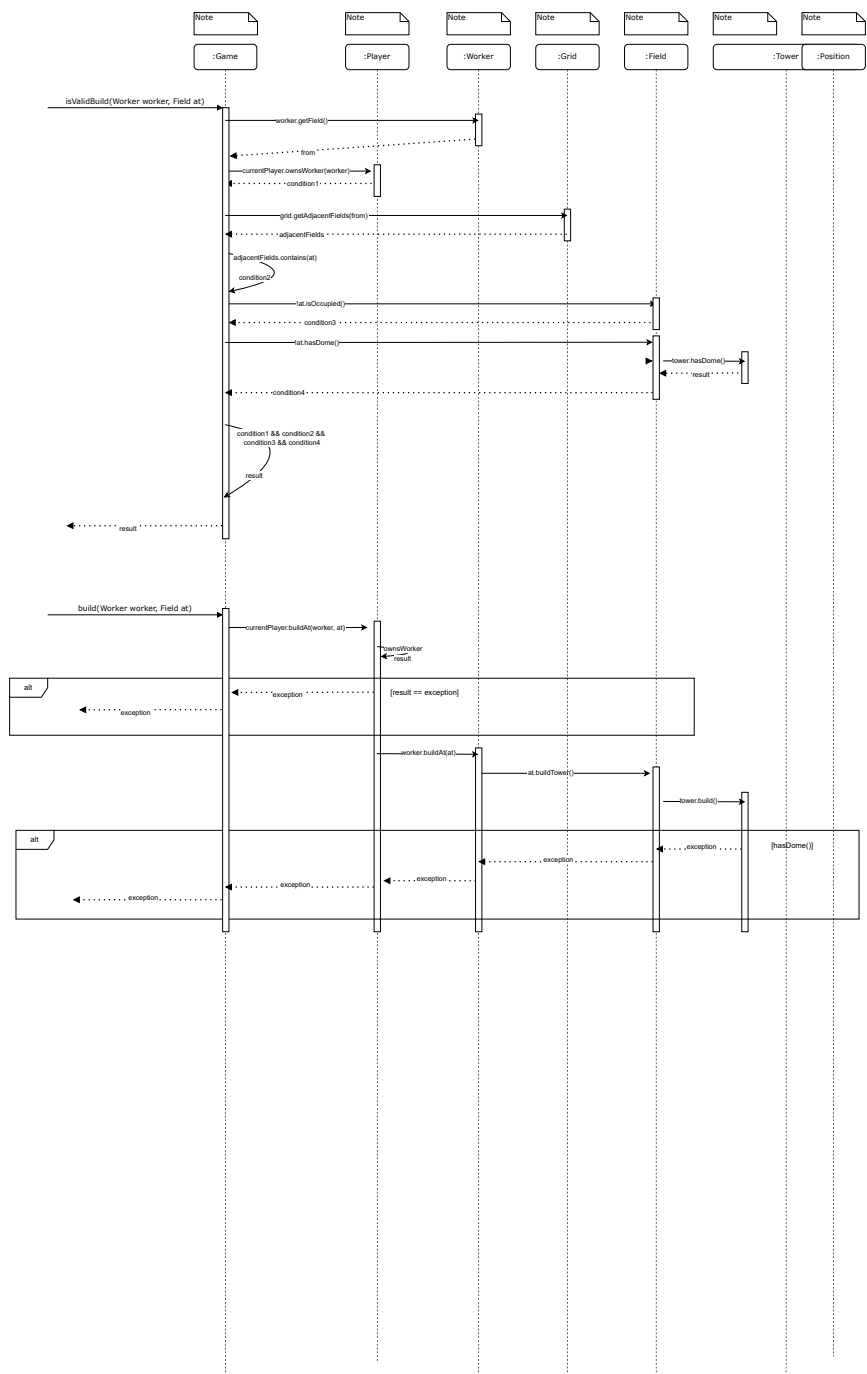




Justification for Building Action

Determining a Valid Build

Where it's checked: `Game` determines if a build is valid using `isValidBuild(Worker worker, Field at)`. **How it's checked:** `isValidBuild(Worker worker, Field at)` determines if a build is valid by checking if `worker` is owned by `currentPlayer`, if `at` is occupied, if `at` has a dome on its `Tower`, and if `at` is adjacent to `worker's` current `Field`.

Justification:

- **Single Responsibility:** `Player` and `Worker` only know which `Field` they are on. They do not track adjacent positions or tower heights. This avoids giving them logic they don't need.
- **Information Expert:** `Game` knows the turn order and has access to `Grid`, allowing it to check adjacent fields without exposing `Grid's` internal structure to `Player`.

Alternatives considered:

1. **Alternative:** `Player` validates builds
- **Trade-off:** `Player` would need access to board layout and game rules, adding unnecessary coupling between `Player` and `Grid` and violating encapsulation, respectively.
2. **Alternative:** `Worker` validates builds - **Trade-off:** Similar drawbacks as with `Player`.

Performing the Build

Where it's executed: `Game` builds with the `build(Worker worker, Field at)` method.

How it's executed: `Game` calls `worker.buildAt(at)`, which delegates to `Field.buildTower()`, which delegates to `Tower.build()` to increase its height, add a dome, or throw an exception if a dome already exists.

Justification:

- **Encapsulation:** `Game` enforces the build rules, but the actual modification of the board is delegated to the game components (`Worker`, `Field`, `Tower`). -
- Information Expert:** `Field` already tracks `Tower`, so it makes sense for `Field` to handle modifications rather than having `Game` or `Worker` do so directly. (In a previous iteration, I violated Law of Demeter with chains of calls. I fixed that in my fix).

Alternatives considered:

1. **Alternative:** `Worker` directly calls `at.buildTower()`
- **Trade-off:** This would bypass `Game's` validation step and allow `Worker` to modify `Field` directly, violating encapsulation.

q